When running the aforementioned test case, Hibernate generates the following output:

```
-- Alice selects the Repository entity
SELECT r.id AS id1_2_0_, r.name AS name2_2_0_, r.version AS version3_2_0_
FROM   repository r
WHERE  r.id = 1

-- Bob selects the Repository entity
SELECT r.id AS id1_2_0_, r.name AS name2_2_0_, r.version AS version3_2_0_
FROM   repository r
WHERE  r.id = 1

-- Bob adds a new Commit entity
INSERT INTO commit (repository_id, id) VALUES (1, 2)

INSERT INTO commit_change (commit_id, diff, path)
VALUES (2, '0a1,2...', 'Intro.md')

-- Bob increments the Repository version
UPDATE repository SET version = 1 WHERE id = 1 AND version = 0

-- Alice adds a new Commit entity
INSERT INTO commit (repository_id, id) VALUES (1, 3)

INSERT INTO commit_change (commit_id, diff, path)
VALUES (3, '0a1,5...', 'FrontMatter.md')

INSERT INTO commit_change (commit_id, diff, path)
VALUES (3, '17c17...', 'HibernateIntro.md')

-- Alice increments the Repository version
UPDATE repository SET version = 1 WHERE id = 1 AND version = 0

--Exception thrown
javax.persistence.RollbackException: Error while committing the transaction
Caused by: javax.persistence.OptimisticLockException:
Caused by: org.hibernate.StaleObjectStateException:
Row was updated or deleted by another transaction
(or unsaved-value mapping was incorrect)
```

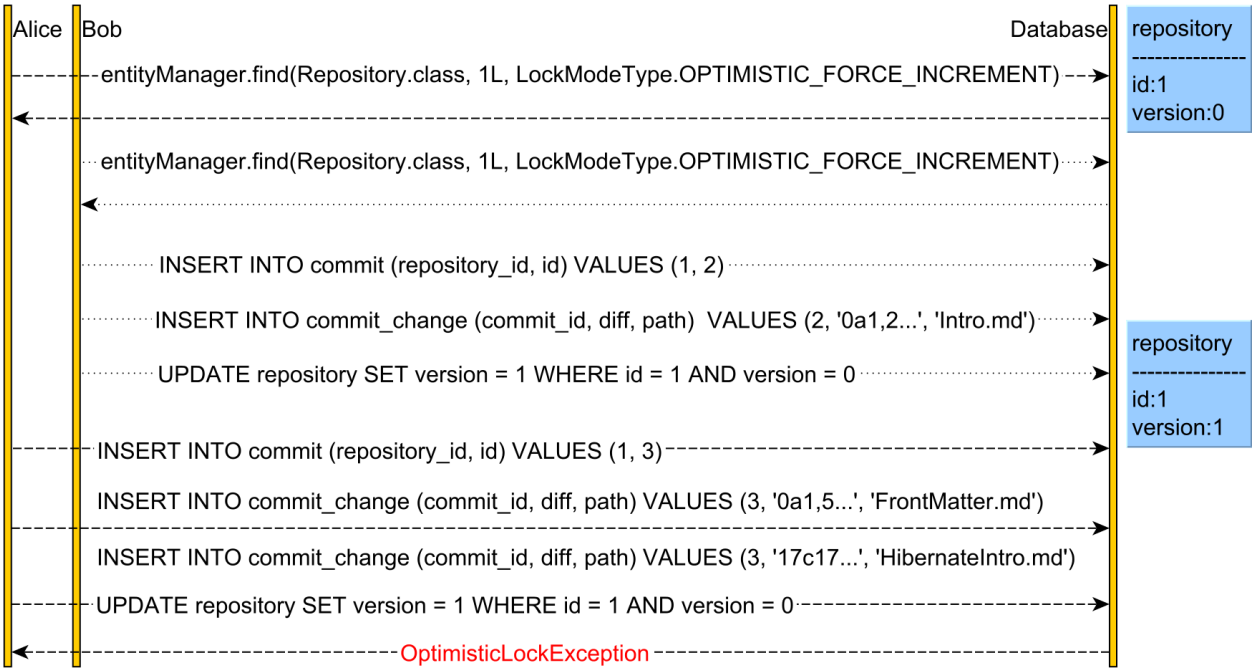

Figure 16.11: LockModeType.OPTIMISTIC_FORCE_INCREMENT

The flow can be explained as follows:

- Alice fetches the `Repository` entity and instructs Hibernate to acquire an `OPTIMISTIC_FORCE_INCREMENT` application-level lock.
- Alice's thread is suspended by the JVM thread scheduler, so Bob gets the chance to fetch the `Repository` entity using the `OPTIMISTIC_FORCE_INCREMENT` lock strategy.
- Bob manages to add a new `Commit` entity, and his transaction is committed. Therefore, the `Repository` entity version is also incremented.
- Alice thread is resumed, and she adds one `Commit` entity and initiates a transaction commit.
- The optimistic locking update fails because the `Repository` version has changed.

`OPTIMISTIC_FORCE_INCREMENT` is useful for propagating a child entity state change to the parent entity optimistic locking version. By applying an optimistic lock on a common parent entity, it is, therefore, possible to coordinate multiple child entities whose changes need to be applied sequentially so that no update is being lost.

16.2.4 LockModeType.PESSIMISTIC_FORCE_INCREMENT

Just like `OPTIMISTIC_FORCE_INCREMENT`, `PESSIMISTIC_FORCE_INCREMENT` can be used to increment the version of any given entity. However, if for `OPTIMISTIC_FORCE_INCREMENT` the entity version is incremented towards the end of the currently running transaction, the `PESSIMISTIC_FORCE_INCREMENT` forces the version incrementation right away, as demonstrated by the test case below.

```
Repository repository = entityManager.find(Repository.class, 1L,
        LockModeType.PESSIMISTIC_FORCE_INCREMENT);

Commit commit = new Commit(repository);
commit.getChanges().add(new Change("FrontMatter.md", "0a1,5..."));
commit.getChanges().add(new Change("HibernateIntro.md", "17c17..."));

entityManager.persist(commit);
```

Hibernate generates the following statements:

```
SELECT r.id AS id1_2_0_, r.name AS name2_2_0_, r.version AS version3_2_0_
FROM   repository r
WHERE  r.id = 1
FOR UPDATE

UPDATE repository SET version = 1 WHERE id = 1 AND version = 0

INSERT INTO commit (repository_id, id) VALUES (1, 2)

INSERT INTO commit_change (commit_id, diff, path)
VALUES (2, '0a1,5...', 'FrontMatter.md')

INSERT INTO commit_change (commit_id, diff, path)
VALUES (2, '17c17...', 'HibernateIntro.md')
```

The `Repository` is locked using a `FOR UPDATE` SQL clause in the select statement that fetches the aforementioned entity. The `Repository` entity is also incremented before the entity is returned to the data access layer.

In the following example, Alice is going to lock the `Repository` only after she previously fetched the very same entity. However, in the meanwhile, Bob is going to increment the `Repository` entity version using a `PESSIMISTIC_FORCE_INCREMENT` lock.

```
Repository repository = entityManager.find(Repository.class, 1L);

executeSync(() -> {
    doInJPA(_entityManager -> {
        Repository _repository = _entityManager.find(Repository.class, 1L,
            LockModeType.PESSIMISTIC_FORCE_INCREMENT);

        Commit _commit = new Commit(_repository);
        _commit.getChanges().add(new Change("Intro.md", "0a1,2..."));
        _entityManager.persist(_commit);
    });
});

entityManager.lock(repository, LockModeType.PESSIMISTIC_FORCE_INCREMENT);
```

When running the test case above, Hibernate generates the following output:

```
-- Alice selects the Repository entity
SELECT r.id AS id1_2_0_, r.name AS name2_2_0_, r.version AS version3_2_0_
FROM   repository r
WHERE  r.id = 1

-- Bob selects the Repository entity
SELECT r.id AS id1_2_0_, r.name AS name2_2_0_, r.version AS version3_2_0_
FROM   repository r
WHERE  r.id = 1
FOR UPDATE

-- Bob increments the Repository version upon fetching the Repository entity
UPDATE repository SET version = 1 WHERE id = 1 AND version = 0

-- Bob adds a new Commit entity
INSERT INTO commit (repository_id, id) VALUES (1, 2)

INSERT INTO commit_change (commit_id, diff, path)
VALUES (2, '0a1,2...', 'Intro.md')

-- Alice tries to increment the Repository entity version
UPDATE repository SET version = 1 WHERE id = 1 AND version = 0

-- Exception thrown
javax.persistence.OptimisticLockException:
Caused by: org.hibernate.StaleObjectStateException:
```

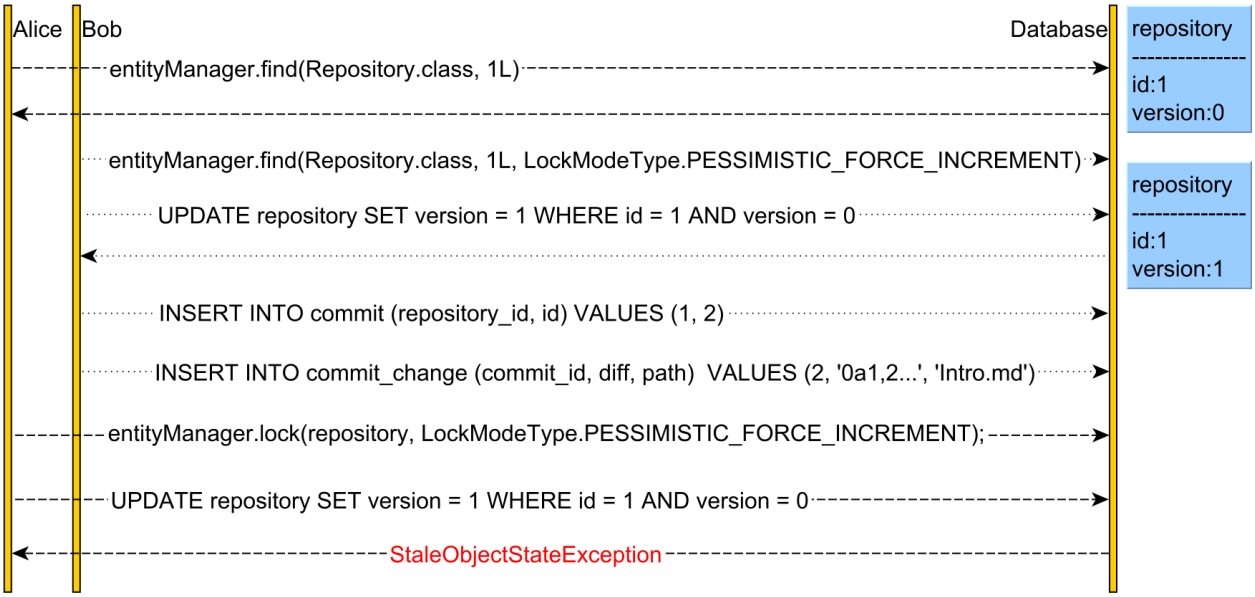

Figure 16.12: Fail fast PESSIMISTIC_FORCE_INCREMENT lock acquisition

The flow can be explained as follows:

- Alice fetches the `Repository` entity without acquiring any physical or logical lock.
- Alice's thread is suspended by the JVM thread scheduler, so Bob gets the chance to fetch the `Repository` entity using the `PESSIMISTIC_FORCE_INCREMENT` lock strategy.
- The `Repository` entity version is incremented right away.
- Bob manages to add a new `Commit` entity, and his transaction is committed.
- Alice thread is resumed, and she attempts to acquire a `PESSIMISTIC_FORCE_INCREMENT` on the already fetched `Repository` entity.
- The optimistic locking update fails because the `Repository` version was changed by Bob.

The instantaneous version incrementation has the following benefits:

- Because the entity is locked at the database row level, the entity version incrementation is guaranteed to succeed.
- If the entity was previously loaded without being locked and the `PESSIMISTIC_FORCE_INCREMENT` version update fails, the currently running transaction can be rolled back right away.